

T2.1.E · Scheduler periódico de reconciliación de reservas

Owner: Celestino · 2026-05-11 · branch *pr/t2.1.e-automation-scheduler-reconciliation* · commit local · sin push · sin PR · sin deploy.

Resumen ejecutivo: el Automation Core ya cuenta con un scheduler periódico opt-in que llama *reconciliar_reservas()* cada *N* segundos (default 300). Por default está **deshabilitado** (requiere *AUTOMATION_SCHEDULER_ENABLED=true*). Se monta sobre el AsyncIOScheduler ya existente · no introduce nueva dependencia. Tiene anti-concurrencia doble (lock + *max_instances=1*) y manejo de excepciones que no rompe el loop. **Suite completa 881 tests OK · +14 nuevos · 0 regresión.**

1. Diseño · qué hace y cómo

1.1 Misión del scheduler

El fix T2.1.D.1 mitigó el riesgo de saldo descontado sin reserva visible mediante write-ahead + reconciliación inline en *reservar()*. El caso residual era: **worker que NO reintenta la acción** tras un crash. Hasta T2.1.E la reconciliación quedaba dependiendo de un admin manual. Ahora corre sola en intervalos configurables.

1.2 Componentes

Componente	Responsabilidad
agent/automation/scheduler.py	Helpers de env, tick de trabajo con lock + try/except, <i><i>registrar_automation_jobs()</i> </i> para montar e
agent/scheduler.py	Llama <i><i>registrar_automation_jobs(scheduler)</i></i> antes de <i><i>scheduler.start()</i></i> . Si la llamada lan
agent/main.py	Expone <i><i>GET /admin/automation/scheduler/status</i></i> · solo lectura · <i><i>_verificar_admin</i></i> con HI
tests/test_automation_scheduler.py	14 tests · cubren env vars, parámetros del tick, manejo de excepciones, lock anti-concurrencia, registr

1.3 Flujo de un tick

```
async def tick_reconciliacion_reservas():
    if _lock.locked():
        # ← anti-concurrencia
        _estado['corridas_saltadas_por_lock'] += 1
        return {'status': 'skipped_lock'}
    async with _lock:
        try:
            resultado = await reconciliar_reservas(
                max_edad_segundos=edad_min_segundos(),
                limit=limit_corrida(),
                dry_run=False,
            )
            _estado['corridas_totales'] += 1
            _estado['ultimo_resultado'] = resultado
            return {'status': 'ok', 'resultado': resultado}
        except Exception as e:
            # ← nunca propaga
            _estado['corridas_fallidas'] += 1
            logger.error(...)
            return {'status': 'error', 'error': type(e).__name__}
```

2. Variables de entorno

Variable	Default	Rango	Significado
----------	---------	-------	-------------

AUTOMATION_SCHEDULER_ENABLED	false 1/true/yes/on (case-insensitive) · master switch · OFF por default · prod opta in explícito
AUTOMATION_SCHEDULER_INTERVAL_SECONDS	30 ≤ N ≤ 3600 · clamped Frecuencia del tick.
AUTOMATION_SCHEDULER_RECONCILE_EDAD_SEGUNDOS	N ≥ 30 Edad mínima para marcar <i>preparing</i> como <i>ready</i>
AUTOMATION_SCHEDULER_RECONCILE_LIMIT	500 1 ≤ N ≤ 5000 · clamped Tpe de filas procesadas por tick.

No se tocaron variables de entorno en Render. Producción sigue con default OFF hasta que el owner agregue la env var. El checklist de smoke test producción (T2.1.D verificación post-merge §7) sigue siendo el siguiente paso obligatorio antes de activarlo en prod.

3. Archivos modificados / creados

Archivo	Tipo	Resumen
agent/automation/scheduler.py	NUEVO ~190 ln	Módulo completo · helpers de env, tick, registro, estado.
agent/scheduler.py	MOD ~12 ln	Bloque <i>try/except</i> dentro de <i>iniciar_scheduler()</i> que llama <i>reconciliar_reservas()</i>
agent/main.py	MOD ~15 ln	Endpoint <i>GET /admin/automation/scheduler/status</i> · protegido con <i>require_auth</i>
tests/test_automation_scheduler.py	NUEVO 14 tests	<i>TestEnvConfig</i> (5) · <i>TestTickLlamaReconciliar</i> (2) · <i>TestTickLlamaReconciliarReservas</i> (2)

Sin nuevas migraciones de schema. El scheduler no introduce tablas ni columnas · sólo consume *reconciliar_reservas()* ya disponible desde T2.1.D.1.

4. Riesgos y mitigaciones

Riesgo	Severidad	Mitigación
Activar el scheduler en prod sin verificar el sm	Bajo	Default OFF. El owner debe agregar la env var en Render para encenderlo · cambio
Tick toma demasiado y se bloquean otros jobs	Bajo	<code>reconciliar_reservas()</code> respeta <code>limit</code> (default 500) · cada operación es le
Dos ticks simultáneos (replicas o reintento de	Bajo	Doble defensa: APScheduler <code>max_instances=1</code> + <code>asyncio.Lock</code> en proc
Excepción del tick rompe el scheduler	Muy bajo	Todo el cuerpo del tick está en <code>try/except Exception</code> . Test <code>test_excepcion_r</code>
Registro del job tumba el startup del web serv	Muy bajo	<code>registrar_automation_jobs()</code> está envuelto en <code>try/except</code> . Si la env var e
Logs filtran PII (teléfonos, ids)	Muy bajo	El tick solo loguea conteos (<code>promoted</code> , <code>marked_failed</code> , <code>intactas</code>).

5. Verificaciones · comandos y resultado

```
pytest tests/test_automation_scheduler.py -v
→ 14 passed in 10.14s

pytest tests/test_automation_*.py
→ 219 passed in (suite automation, incluye 14 nuevos)

pytest
→ 881 passed in 237.12s · +14 vs main · 0 failed · 0 regresión
```

6. Cómo validar localmente

6.1 Sin tocar prod · ejecutar la suite

```
git checkout pr/t2.1.e-automation-scheduler-reconciliation
pytest tests/test_automation_scheduler.py -v
pytest                                # suite completa
```

6.2 Smoke manual del scheduler en dev (opt-in)

Solo si quieres ver el scheduler corriendo realmente · **en local** · **NO en prod**:

```
# Terminal · arrancar Dona con el scheduler activo
set AUTOMATION_SCHEDULER_ENABLED=true
set AUTOMATION_SCHEDULER_INTERVAL_SECONDS=30
uvicorn agent.main:app --reload --port 8080

# En otro terminal · esperar 1 minuto · luego consultar estado
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
  http://localhost:8080/admin/automation/scheduler/status
→ {
  "habilitado": true,
  "intervalo_segundos": 30,
  "corridas_totales": 2,
  "corridas_saltadas_por_lock": 0,
  "corridas_fallidas": 0,
  "ultima_corrida_inicio": "2026-05-11T...",
  "ultimo_resultado": {
    "dry_run": false,
    "total_preparing_inicial": 0,
    "promoted": 0,
    "marked_failed": 0,
    "intactas": 0
  }
}
```

6.3 Verificar logs

```
# Esperado en stdout del uvicorn:
[AUT-SCHED] reconciliación registrada · intervalo=30s · max_edad=120s · limit=500
[AUT-SCHED] reconciliación OK · intactas=0      # cada tick
```

7. Estado git

Item	Valor
Branch	<i>pr/t2.1.e-automation-scheduler-reconciliation</i>
Base	<i>main</i> · sincronizada antes del branch (commit <i>f4f258e</i>)
Commit en branch	Por crear · <i>feat(automation): scheduler periódico de reconciliación de reservas (T2.1.E)</i>
Push	No realizado · esperando aprobación
PR	No abierto · per instrucción
Deploy	No realizado

8. Siguiente paso recomendado

Cuando estés listo para mover T2.1.E adelante:

#	Acción	Riesgo
1	Revisar el diff local · <i>git diff main..HEAD</i>	Cero
2	Push de la branch: <i>git push -u origin pr/t2.1.e-automation-scheduler-reconciliation</i>	Cero (branch nueva, sin afectar main)
3	Abrir PR con título: <i>feat(automation): periodic scheduler for reservations reconciliation (T2.1.E)</i>	Cero (T2.1.E) merge
4	Mergear vía PR cuando el smoke test producción de T2.1.D pase (ver Resumen_T2.1.D y el Post-Merge de T2.1.D)	Bajo (verificado Post-Merge de T2.1.D)
5	Decidir cuándo activar en prod: agregar <i>AUTOMATION_SCHEDULER_ENABLED=true</i> en Render. Recomendación: dejar el env var	Bajo
6	NO ahora: T2.2 (acción HIGH real con Whapi). Esperar a que T2.1.E haya corrido 1 semana en prod sin <i>corridas_fallidas > 0</i>	Bajo

Sin acciones automáticas pendientes. El scheduler está implementado, testeado y listo en una branch local. Producción no se ve afectada hasta que (a) el PR se mergee y (b) la env var se active explícitamente.